

JAVA COLLECTIONS FRAMEWORK

Day 1 — Collection Framework Overview & List Classes

Interview-Ready Notes · Internal Working · Code · Mock Interview

ArrayList

LinkedList

Vector

Stack

Iterable vs Collection

CRUD + Iteration

WHAT'S INSIDE

- Internal working of every class
- Real-world examples
- Complete runnable code
- 35-40 interview questions
- Coding problems with solutions
- AI-style interview questions
- Full mock interview transcript
- Comparison tables & cheat sheets

Prepared as a self-study & revision guide · Java SE

Table of Contents

1. Collection Framework Overview	Theory
2. Arrays vs Collections	Theory
3. Iterable vs Collection	Theory
4. Collection vs Collections	Theory
5. List Interface Overview	Theory
6. ArrayList — Internal Working, Example, Code	Theory
7. LinkedList — Internal Working, Example, Code	Theory
8. Vector — Internal Working, Example, Code	Theory
9. Stack — Internal Working, Example, Code	Theory
10. CRUD Operations (Full Code)	Coding
11. Iteration — All Techniques	Coding
12. Student Management System (Mini Project)	Coding
13. 35-40 Interview Questions & Answers	Interview
14. Coding Problems	Interview
15. AI-Style Interview Questions	Interview
16. Mock Interview Transcript	Interview

Theory — Collection Framework Overview

1. Collection Framework Overview

The **Java Collections Framework (JCF)** is a unified architecture of interfaces and classes used to store, retrieve, manipulate, and communicate aggregate data. It was introduced in **Java 1.2** and lives in the `java.util` package.

Internal Working

The framework is built around a small set of core **interfaces** (`Collection`, `List`, `Set`, `Queue`, `Map`) that define *what* operations are possible, and a set of **implementation classes** (`ArrayList`, `LinkedList`, `HashSet`, `HashMap`, etc.) that define *how* those operations are carried out internally (arrays, linked nodes, hash tables, trees). This separation of interface from implementation is what lets you swap `List<String> list = new ArrayList<>();` for `new LinkedList<>();` without changing any other code — this is **programming to an interface**.

Framework Hierarchy (simplified)

```
Iterable (interface)
├── Collection (interface)
│   ├── List      → ArrayList, LinkedList, Vector, Stack
│   ├── Set       → HashSet, LinkedHashSet, TreeSet
│   └── Queue     → PriorityQueue, ArrayDeque, LinkedList
└── Map (separate hierarchy, NOT a Collection)
    ├── HashMap, LinkedHashMap, TreeMap, Hashtable
```

Real-world Example

Think of an e-commerce cart: the **shopping cart** (an ordered, duplicate-allowing list of items) maps naturally to a `List`; the set of **unique coupon codes** applied maps to a `Set`; and the **product catalog** keyed by product ID maps to a `Map`. Choosing the right collection is choosing the right real-world data shape.

2. Arrays vs Collections

Aspect	Array	Collection
Size	Fixed at creation time	Grows/shrinks dynamically
Data type	Can hold primitives & objects	Can hold only objects (autoboxing for primitives)
Utility methods	No built-in add/remove/search helpers	Rich API — <code>add()</code> , <code>remove()</code> , <code>contains()</code> , <code>sort()</code> , etc.
Memory	Contiguous block, allocated once	Managed internally, may resize/copy
Multi-dimensional	Natively supported	Simulated via nested collections
Performance	Faster for fixed-size numeric data	Slight overhead due to abstraction & boxing

Internal Working

An array is a raw, contiguous memory block of a fixed length decided at instantiation (`new int[10]`). Collections like `ArrayList` are built *on top of* arrays internally, but they add a resizing algorithm: when the backing array is full, a new, larger array is allocated (typically 1.5× the old size) and elements are copied over — giving the illusion of dynamic growth.

3. Iterable vs Collection

Aspect	Iterable	Collection
Package	java.lang	java.util
Position	Top-most interface (grandparent)	Extends Iterable (parent of List/Set/Queue)
Purpose	Only guarantees the object can be iterated (for-each loop)	Adds actual storage operations: add, remove, size, contains
Abstract method	<code>Iterator<T> iterator()</code>	Inherits iterator() + adds ~15 more methods
Used by	Collection, and also custom classes just for for-each support	List, Set, Queue

```
public interface Iterable<T> {
    Iterator<T> iterator(); // the ONLY abstract method
}

public interface Collection<E> extends Iterable<E> {
    boolean add(E e);
    boolean remove(Object o);
    int size();
    boolean isEmpty();
    boolean contains(Object o);
    // ...many more
}
```

Tip

Any class that implements `Iterable` — even if it is *not* a `Collection` — can be used in an enhanced for-loop. That's why `Map` is not a `Collection`, yet `map.entrySet()` (which IS a `Collection`) can be looped over directly.

4. Collection vs Collections

Aspect	Collection	Collections
What it is	An interface	A final utility class
Location	Root of List/Set/Queue hierarchy	java.util.Collections
Contains	Abstract method declarations	Static helper methods only
Example use	<code>Collection<String> c = new ArrayList<>();</code>	<code>Collections.sort(list); Collections.reverse(list);</code>
Instantiable?	No (interface)	No (private constructor)

Real-world Example

`Collection` is like the blueprint "container" concept (a box). `Collections` is like a toolbox full of ready-made utilities (a tape measure, a label maker) you use *on* boxes — e.g. `Collections.sort()`, `Collections.max()`, `Collections.unmodifiableList()`, `Collections.synchronizedList()`.

5. List Interface (Overview)

`List` is an **ordered** collection (also called a *sequence*) that allows **duplicate elements** and gives you **index-based access**. It extends `Collection` and adds position-aware methods.

Key List methods	Purpose
<code>get(int index)</code>	Retrieve element by position
<code>set(int index, E e)</code>	Replace element at a position
<code>add(int index, E e)</code>	Insert at a specific position
<code>indexOf(Object o)</code>	First index of an element
<code>listIterator()</code>	Bidirectional iterator
<code>subList(from, to)</code>	View of a portion of the list

Major implementations: ArrayList, LinkedList, Vector, Stack (each covered in detail below).

ArrayList

6. ArrayList

`ArrayList` is a resizable-array implementation of the `List` interface. It is the most commonly used `List` because it offers fast random access and a familiar array-like feel with automatic resizing.

Internal Working

Internally, `ArrayList` stores elements in a plain `Object[] elementData` array. A no-arg constructor starts with an **empty array** (capacity 0) and lazily allocates a default capacity of **10** on the first `add()`. When the array is full, `grow()` is called, which allocates a new array of size `oldCapacity + (oldCapacity >> 1)` — i.e. **1.5× the old capacity** — and copies every element into it using `Arrays.copyOf()`. This makes `add()` **O(1) amortized** but occasionally O(n) during a resize. `get(index)` and `set(index, e)` are O(1) because they're direct array-index lookups. `add(index, e)` and `remove(index)` are O(n) because all subsequent elements must be shifted using `System.arraycopy()`. `ArrayList` is **not synchronized** — multiple threads modifying it concurrently can corrupt state, and modifying it during iteration (except via the iterator's own `remove()`) throws `ConcurrentModificationException` via the `modCount` fail-fast mechanism.

Real-world Example

A **to-do list app** where tasks are shown in the order added and you frequently scroll/jump to a specific task by position (e.g. "show me task #4") — fast random access via index makes `ArrayList` ideal. Similarly, a music player's **current playlist**.

Complete Code Example

```
import java.util.ArrayList;
import java.util.List;

public class ArrayListDemo {
    public static void main(String[] args) {
        List<String> fruits = new ArrayList<>();

        fruits.add("Apple");           // O(1) amortized
        fruits.add("Banana");
        fruits.add(1, "Mango");        // insert at index 1 -> O(n)

        System.out.println(fruits);    // [Apple, Mango, Banana]
        System.out.println(fruits.get(2)); // Banana -> O(1)

        fruits.set(0, "Green Apple");  // O(1)
        fruits.remove("Mango");        // O(n) - search + shift
        fruits.remove(0);              // O(n) - shift

        System.out.println("Size: " + fruits.size());
        System.out.println("Contains Banana? " + fruits.contains("Banana"));
    }
}
```

Operation	Time Complexity	Why
<code>get(index)</code> / <code>set(index)</code>	O(1)	Direct array index access
<code>add(element)</code> at end	O(1) amortized	Occasional resize copies all elements
<code>add(index, element)</code>	O(n)	Shifts elements right
<code>remove(index)</code>	O(n)	Shifts elements left
<code>contains()</code> / <code>indexOf()</code>	O(n)	Linear scan

7. LinkedList

`LinkedList` implements both `List` and `Deque`, making it a doubly-linked-list-based sequence that is efficient for insertions/deletions at the ends and for use as a stack or queue.

Internal Working

`LinkedList` is a **doubly linked list** of private static `Node<E>` objects, each holding a reference to `item`, `next`, and `prev`. The class keeps direct references to the `first` and `last` nodes, so operations at either end — `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()` — are **$O(1)$** because no shifting is needed, just pointer re-linking. However, `get(index)` is **$O(n)$** because the list must be walked node-by-node from whichever end is closer. There is no resizing/copying cost like `ArrayList` since each node is allocated independently — but each node carries extra memory overhead (two pointer references per element) compared to a plain array slot.

Real-world Example

A **music player queue** ("play next" / "add to front") or a **browser's back/forward history**, where you constantly add/remove at the front or back but rarely jump to an arbitrary middle position by index.

Complete Code Example

```
import java.util.LinkedList;

public class LinkedListDemo {
    public static void main(String[] args) {
        LinkedList<String> songs = new LinkedList<>();

        songs.add("Song A");
        songs.addLast("Song B");
        songs.addFirst("Song C");           //  $O(1)$  - pointer update only

        System.out.println(songs);          // [Song C, Song A, Song B]

        songs.removeFirst();                 //  $O(1)$ 
        songs.removeLast();                  //  $O(1)$ 

        System.out.println("Peek: " + songs.peek()); // head element
        songs.push("Song D");                // Deque method - adds to front
        System.out.println(songs);
    }
}
```

Feature	ArrayList	LinkedList
Backing structure	Dynamic array	Doubly linked nodes
<code>get(index)</code>	$O(1)$	$O(n)$
add/remove at ends	$O(1)$ end / $O(n)$ front	$O(1)$ both ends
add/remove middle	$O(n)$	$O(n)$ to find + $O(1)$ to link
Memory overhead	Low (just array slots)	Higher (2 pointers/node)
Implements	List, RandomAccess	List, Deque, Queue

8. Vector

`Vector` is a legacy (Java 1.0) resizable array implementation, functionally similar to `ArrayList` but **synchronized**.

Internal Working

Like `ArrayList`, `Vector` stores data in an internal `Object[]` array. The key difference is that **every public method is `synchronized`** (e.g. `add()`, `get()`, `remove()` all acquire an intrinsic lock on the `Vector` object), making it thread-safe at the level of individual method calls. Its default growth strategy also differs slightly: unless a `capacityIncrement` is specified, it **doubles** its capacity (100% growth) on resize, versus `ArrayList`'s 50% growth. The synchronization overhead makes `Vector` noticeably slower than `ArrayList` in single-threaded code, and it still doesn't guarantee atomicity across *multiple* operations (e.g. check-then-add is not atomic) — for that you'd need external locking or `CopyOnWriteArrayList` / `Collections.synchronizedList` with proper block-synchronization.

Real-world Example

Legacy enterprise systems (pre-Java-1.2 codebases, or older Swing/AWT UI code) that need a simple thread-safe list without pulling in `java.util.concurrent`. In modern code, `Vector` is largely replaced by `Collections.synchronizedList(new ArrayList<>())` or `CopyOnWriteArrayList`.

Complete Code Example

```
import java.util.Vector;

public class VectorDemo {
    public static void main(String[] args) {
        Vector<Integer> scores = new Vector<>();

        scores.add(90);
        scores.add(85);
        scores.addElement(78);    // legacy-style method, same as add()

        System.out.println(scores);    // [90, 85, 78]
        System.out.println("Capacity behavior differs: doubles on grow");

        synchronized (scores) {      // still needed for compound actions
            if (!scores.contains(100)) {
                scores.add(100);
            }
        }
        System.out.println(scores);
    }
}
```

9. Stack

`Stack` extends `Vector` and implements the classic **LIFO (Last-In-First-Out)** data structure.

Internal Working

`Stack` literally **extends `Vector<E>`**, so it inherits `Vector`'s synchronized array-backed storage. It adds five methods on top: `push(e)` (calls `addElement`, appends to the end of the underlying array), `pop()` (removes & returns the last element), `peek()` (returns the last element without removing), `empty()`, and `search(e)` (returns 1-based distance from the top). Because it's built on an array via `Vector`, push/pop at the "top" (which is really the array's end) are $O(1)$ amortized. Oracle's own docs recommend using `ArrayDeque` instead of `Stack` for stack semantics today, since `Stack` inherits many un-stack-like List methods (like `insertElementAt`) that break LIFO discipline, plus it carries `Vector`'s synchronization overhead.

Real-world Example

The **"undo" feature** in a text editor (each action pushed on a stack, undo = pop), a **browser's back-button history** in single-direction form, or evaluating balanced parentheses / expression evaluation in a compiler.

Complete Code Example

```
import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {
        Stack<String> undoStack = new Stack<>();

        undoStack.push("Type 'Hello'");
        undoStack.push("Bold text");
        undoStack.push("Insert image");

        System.out.println("Top action: " + undoStack.peek()); // Insert image

        while (!undoStack.isEmpty()) {
            System.out.println("Undo: " + undoStack.pop());
        }
        // Output order: Insert image -> Bold text -> Type 'Hello' (LIFO)
    }
}
```

Class	Thread-safe?	Growth	Best used as
ArrayList	No	1.5×	General-purpose list, fast random access
LinkedList	No	N/A (node-based)	Frequent add/remove at ends, Deque/Queue
Vector	Yes (synchronized)	2×	Legacy thread-safe list
Stack	Yes (extends Vector)	2×	LIFO operations (prefer ArrayDeque today)

Coding — CRUD, Iteration & Mini Project

10. CRUD Operations

CRUD = **Create, Read, Update, Delete**. Below is a complete demonstration using `ArrayList`, applicable the same way to `LinkedList`/`Vector`.

```
import java.util.ArrayList;
import java.util.List;

public class CrudDemo {
    public static void main(String[] args) {
        List<String> cities = new ArrayList<>();

        // CREATE
        cities.add("Hyderabad");
        cities.add("Bengaluru");
        cities.add("Chennai");
        System.out.println("After Create: " + cities);

        // READ
        String first = cities.get(0);
        System.out.println("Read index 0: " + first);
        for (String city : cities) {
            System.out.println("Reading -> " + city);
        }

        // UPDATE
        cities.set(1, "Mumbai");    // replace Bengaluru with Mumbai
        System.out.println("After Update: " + cities);

        // DELETE
        cities.remove("Chennai");   // remove by value
        cities.remove(0);           // remove by index
        System.out.println("After Delete: " + cities);
    }
}
```

CRUD	List Method(s)
Create	<code>add(e)</code> , <code>add(index, e)</code>
Read	<code>get(index)</code> , iteration, <code>contains()</code>
Update	<code>set(index, e)</code>
Delete	<code>remove(index)</code> , <code>remove(Object)</code> , <code>clear()</code>

11. Iteration — All Techniques

```
import java.util.*;

public class IterationDemo {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 40));

        // 1) Classic for loop (index-based, needs RandomAccess for efficiency)
        for (int i = 0; i < nums.size(); i++) {
            System.out.println("for: " + nums.get(i));
        }

        // 2) Enhanced for-each (uses Iterator internally)
        for (int n : nums) {
            System.out.println("for-each: " + n);
        }

        // 3) Iterator - safe removal during iteration
        Iterator<Integer> it = nums.iterator();
        while (it.hasNext()) {
            int val = it.next();
            if (val == 20) it.remove(); // safe; avoids ConcurrentModificationException
        }

        // 4) ListIterator - bidirectional traversal
        ListIterator<Integer> lit = nums.listIterator();
        while (lit.hasNext()) lit.next();
        while (lit.hasPrevious()) {
            System.out.println("backwards: " + lit.previous());
        }

        // 5) forEach with lambda (Java 8+)
        nums.forEach(n -> System.out.println("lambda: " + n));

        // 6) Stream API (Java 8+)
        nums.stream().filter(n -> n > 10).forEach(System.out::println);
    }
}
```

Tip — Why use `Iterator.remove()` instead of `list.remove()` in a loop?

Modifying a list directly (e.g. calling `list.remove()`) while iterating with a for-each loop throws `ConcurrentModificationException` because the loop's internal `Iterator` checks a `modCount` field that gets out of sync. `Iterator.remove()` updates that field correctly and is the only safe way to remove elements mid-iteration.

12. Student Management System (Mini Project)

A small console application demonstrating CRUD + iteration on a custom object using `ArrayList<Student>`.

```

import java.util.ArrayList;
import java.util.List;
import java.util.Iterator;

class Student {
    int id;
    String name;
    double marks;

    Student(int id, String name, double marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    @Override
    public String toString() {
        return "[" + id + "] " + name + " - " + marks + "%";
    }
}

public class StudentManagementSystem {
    private List<Student> students = new ArrayList<>();

    // CREATE
    void addStudent(int id, String name, double marks) {
        students.add(new Student(id, name, marks));
    }

    // READ - single
    Student findById(int id) {
        for (Student s : students) {
            if (s.id == id) return s;
        }
        return null;
    }

    // READ - all
    void displayAll() {
        System.out.println("---- Student Records ----");
        for (Student s : students) {
            System.out.println(s);
        }
    }

    // UPDATE
    boolean updateMarks(int id, double newMarks) {
        Student s = findById(id);
        if (s != null) {
            s.marks = newMarks;
            return true;
        }
        return false;
    }

    // DELETE
    boolean removeStudent(int id) {
        Iterator<Student> it = students.iterator();
        while (it.hasNext()) {
            if (it.next().id == id) {
                it.remove();
                return true;
            }
        }
        return false;
    }

    // Extra: average marks (business logic on top of collection)
    double averageMarks() {
        double total = 0;
        for (Student s : students) total += s.marks;
        return students.isEmpty() ? 0 : total / students.size();
    }

    public static void main(String[] args) {
        StudentManagementSystem sms = new StudentManagementSystem();
    }
}

```

```

        sms.addStudent(1, "Aarav", 88.5);
        sms.addStudent(2, "Diya", 92.0);
        sms.addStudent(3, "Kabir", 76.3);

        sms.displayAll();

        sms.updateMarks(3, 81.0);
        System.out.println("\nAfter updating Kabir's marks:");
        sms.displayAll();

        sms.removeStudent(2);
        System.out.println("\nAfter removing Diya:");
        sms.displayAll();

        System.out.printf("\nClass average: %.2f\n", sms.averageMarks());
    }
}

```

Real-world Example

This is the exact pattern behind many college ERP portals and internal HR tools: an in-memory (or DB-backed) list of records, with CRUD operations exposed through a service layer — the same shape you'll build with Spring Boot + JPA later, just using a database table instead of an `ArrayList`.

Interview Questions (35-40)

A. Collection Framework Basics

1 What is the Java Collections Framework?

A unified set of interfaces (Collection, List, Set, Queue, Map) and implementation classes in `java.util` for storing and manipulating groups of objects, introduced in Java 1.2.

2 Why doesn't Map extend Collection?

Map stores key-value pairs, not single elements, so its method contracts (like `put(key, value)`) don't fit the single-element contract of `Collection.add(e)`. It's a deliberately separate hierarchy, though `map.keySet()`, `values()`, and `entrySet()` each return real Collection views.

3 What is the difference between Iterable and Collection?

Iterable (java.lang) only guarantees a for-each-capable `iterator()` method. Collection (java.util) extends Iterable and adds storage operations like add, remove, size, contains.

4 What is the difference between Collection and Collections?

Collection is a root interface; Collections is a final utility class of static helper methods (sort, reverse, synchronizedList, unmodifiableList, etc.).

5 How is an Array different from an ArrayList?

Arrays have a fixed size and can hold primitives directly; ArrayList is dynamically resizable, holds only objects (autoboxed primitives), and offers a rich utility API.

B. List Interface & ArrayList

6 What makes List different from Set?

List is ordered, index-based, and allows duplicates. Set does not allow duplicates and (in most implementations) doesn't guarantee index-based access.

7 How does ArrayList grow internally?

It starts with capacity 0 (or 10 on first add), and when full, allocates a new array of 1.5× the old capacity via `Arrays.copyOf()`, copying all elements across.

8 What is the time complexity of get(), add(), and remove() in ArrayList?

`get()/set()` are $O(1)$. `add()` at the end is $O(1)$ amortized. `add(index)/remove(index)` are $O(n)$ due to shifting.

9 Is ArrayList thread-safe?

No. Concurrent modification from multiple threads can corrupt state or throw `ConcurrentModificationException`. Use `Collections.synchronizedList()` or `CopyOnWriteArrayList` for thread safety.

10 What is ConcurrentModificationException and when does it occur?

A runtime exception thrown when a collection is structurally modified while being iterated by anything other than the iterator's own methods (fail-fast behavior driven by an internal modCount check).

11 How do you make an ArrayList read-only?

Wrap it using `Collections.unmodifiableList(list)`; any mutating call on the returned view throws `UnsupportedOperationException`.

12 What happens if you call new ArrayList<>(50)?

50 is treated as the initial capacity (not size) — an empty list is created with internal array space pre-allocated for 50 elements, avoiding early resizes.

C. LinkedList

13 How is LinkedList implemented internally?

As a doubly linked list of Node objects, each storing the element plus references to the previous and next nodes; the class tracks first and last node references directly.

14 When would you prefer LinkedList over ArrayList?

When you need frequent insertions/deletions at the beginning or end ($O(1)$) and rarely need random access by index, or when using it as a Deque/Queue.

15 Why is get(index) slow on a LinkedList?

There's no direct index-to-memory mapping; the list must be traversed node-by-node from the nearer end, making it $O(n)$.

16 Does LinkedList implement Queue and Deque?

Yes — `LinkedList` implements both `List` and `Deque` (which extends `Queue`), so it supports `addFirst/addLast/offer/poll/push/pop`.

D. Vector & Stack

17 How is Vector different from ArrayList?

`Vector`'s methods are synchronized (thread-safe but slower); it also grows by doubling capacity by default versus `ArrayList`'s $1.5\times$. `Vector` is a legacy class from Java 1.0.

18 Is Vector fully thread-safe for compound operations?

No. Individual method calls are synchronized, but compound actions like "check-then-add" aren't atomic — you still need explicit synchronized blocks around such sequences.

19 How does Stack relate to Vector?

`Stack` extends `Vector`, inheriting its synchronized array-backed storage, and adds `push/pop/peek/empty/search` methods to implement LIFO semantics.

20 Why is ArrayDeque preferred over Stack today?

Stack inherits Vector's List methods (e.g. `insertElementAt`) that can break LIFO discipline, plus unnecessary synchronization overhead. ArrayDeque is faster, unsynchronized, and purpose-built for stack/queue use.

21 What does Stack.search() return?

The 1-based position of an element from the top of the stack, or -1 if not found.

E. Iteration & Comparisons

22 What's the difference between Iterator and ListIterator?

Iterator only moves forward and supports `remove()`. ListIterator (List-specific) can move both forward and backward, and additionally supports `set()` and `add()` during traversal.

23 What is fail-fast vs fail-safe iteration?

Fail-fast iterators (ArrayList, LinkedList, Vector) throw `ConcurrentModificationException` on detecting concurrent structural changes via `modCount`. Fail-safe iterators (e.g. `CopyOnWriteArrayList`) iterate over a cloned snapshot, so they never throw, but may not reflect the latest changes.

24 How does the enhanced for-loop work internally on a List?

The compiler desugars it into calls to `iterator()`, `hasNext()`, and `next()` on the underlying Iterator — it's syntactic sugar, not a different mechanism.

25 Can you add elements to a List while using forEach(lambda)?

No — structurally modifying the list inside a `forEach` lambda still triggers `ConcurrentModificationException`, since `forEach` also uses the iteration/`modCount` mechanism internally.

26 What is RandomAccess and which List classes implement it?

RandomAccess is a marker interface signaling that a List supports fast ($O(1)$) index-based access. ArrayList and Vector implement it; LinkedList does not.

F. Design & Best Practices

27 Why is it recommended to code against the List interface rather than ArrayList directly?

It decouples your code from a specific implementation (programming to an interface), so you can later switch to LinkedList/Vector without touching business logic that only relies on List's contract.

28 How do you sort a List of custom objects?

Implement `Comparable<T>` (natural ordering via `compareTo`) and call `Collections.sort(list)`, or pass a `Comparator` (often a lambda) to `list.sort(comparator)` / `Collections.sort(list, comparator)`.

29 What's the difference between Comparable and Comparator?

Comparable is implemented by the class itself to define one natural ordering (`compareTo`). Comparator is a separate strategy object that can define multiple, external orderings (`compare`) without modifying the class.

30 How would you remove duplicates from an ArrayList?

Convert it to a Set that preserves or discards order as needed, e.g. `new ArrayList<>(new LinkedHashSet<>(list))` to remove duplicates while keeping insertion order.

31 What's the initial default capacity of ArrayList and Vector?

Both default to 10 for a no-arg constructor (ArrayList allocates it lazily on first add; Vector allocates it immediately at construction).

32 How can you make an ArrayList thread-safe without switching to Vector?

Wrap it with `Collections.synchronizedList(new ArrayList<>())` and manually synchronize on the list object during iteration, or use `CopyOnWriteArrayList` for read-heavy, write-light scenarios.

33 What happens internally when you call `remove(int index)` versus `remove(Object o)` on a `List<Integer>`?

`remove(int)` removes by position. `remove(Object)` removes by value (first match via `equals()`). This is a classic gotcha with Integer lists: `list.remove(1)` removes the element at index 1, not the value 1 — you must use `list.remove(Integer.valueOf(1))` to remove by value.

34 Why can't you store primitives directly in a List?

Generics in Java are erased to Object at runtime and only work with reference types, so primitives are autoboxed into their wrapper classes (`int` → `Integer`) before storage, adding some memory/performance overhead.

35 What does `List.of()` return, and how is it different from `new ArrayList<>()`?

`List.of()` (Java 9+) returns an immutable list — `add/remove/set` throw `UnsupportedOperationException` — whereas `new ArrayList<>()` is fully mutable.

36 How do you convert an array to a List and vice versa?

Array → List: `Arrays.asList(arr)` (fixed-size view) or `new ArrayList<>(Arrays.asList(arr))` for a mutable copy. List → array: `list.toArray(new String[0])`.

37 What's the difference between `size()` and `capacity()` in ArrayList context?

`size()` is the number of elements actually stored; `capacity()` is the length of the internal backing array (not exposed by a public method, but relevant to performance/resizing).

38 Can a List contain null values?

ArrayList, LinkedList, and Vector all permit multiple null elements; only some Collection implementations (e.g. certain Queue/Deque implementations, and Map keys in some cases) restrict nulls.

39 How would you reverse a List in place?

`Collections.reverse(list)` reverses the order of elements in place without creating a new list.

40 What's the space-time trade-off between ArrayList and LinkedList?

ArrayList uses less memory per element (just array slots, occasional wasted capacity) but pays for shifting on middle inserts. LinkedList uses more memory per element (two pointers per node) but avoids shifting, paying instead in traversal cost for indexed access.

Coding Problems

Problem 1 — Remove duplicates from an ArrayList while preserving order

```
List<Integer> nums = new ArrayList<>(List.of(4, 2, 4, 5, 2, 8));
List<Integer> unique = new ArrayList<>(new LinkedHashSet<>(nums));
System.out.println(unique);    // [4, 2, 5, 8]
```

LinkedHashSet removes duplicates via equals()/hashCode() while its linked-list-backed bucket ordering preserves insertion order.

Problem 2 — Reverse an ArrayList without using Collections.reverse()

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));
int left = 0, right = list.size() - 1;
while (left < right) {
    int temp = list.get(left);
    list.set(left, list.get(right));
    list.set(right, temp);
    left++;
    right--;
}
System.out.println(list);    // [5, 4, 3, 2, 1]
```

Problem 3 — Implement a stack using two queues (conceptual, using LinkedList as Queue)

```
class StackUsingQueues<T> {
    private LinkedList<T> q1 = new LinkedList<>();
    private LinkedList<T> q2 = new LinkedList<>();

    void push(T x) {
        q2.offer(x);
        while (!q1.isEmpty()) q2.offer(q1.poll());
        LinkedList<T> temp = q1; q1 = q2; q2 = temp;    // swap references
    }

    T pop() { return q1.poll(); }    // front of q1 is always the top
    T top() { return q1.peek(); }
    boolean isEmpty() { return q1.isEmpty(); }
}
```

Tests understanding of Queue/Deque operations and manipulating references between two LinkedList-backed queues.

Problem 4 — Find the second largest element in an ArrayList

```
List<Integer> nums = List.of(12, 45, 2, 41, 45, 30);
int max = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int n : nums) {
    if (n > max) { second = max; max = n; }
    else if (n > second && n != max) { second = n; }
}
System.out.println("Second largest: " + second);    // 41
```

Problem 5 — Merge two sorted ArrayLists into one sorted ArrayList

```
List<Integer> a = List.of(1, 3, 5, 7);
List<Integer> b = List.of(2, 4, 6, 8);
List<Integer> merged = new ArrayList<>(a);
merged.addAll(b);
Collections.sort(merged);
System.out.println(merged);    // [1, 2, 3, 4, 5, 6, 7, 8]
```

Follow-up in interviews: implement this manually with two pointers in $O(n)$ instead of relying on `Collections.sort()` ($O(n \log n)$) — shows you understand the merge step of merge sort.

Problem 6 — Rotate an ArrayList to the right by k positions

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));
int k = 2 % list.size();
Collections.rotate(list, k);
System.out.println(list);    // [4, 5, 1, 2, 3]
```

Problem 7 — Check if a String is a valid balanced-parenthesis expression using a Stack

```
boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char open = stack.pop();
            if ((c == ')' && open != '(') ||
                (c == '}' && open != '{') ||
                (c == ']' && open != '[')) return false;
        }
    }
    return stack.isEmpty();
}
// isValid("{[()]}") -> true
// isValid("[{()}]") -> false
```

A classic Stack use-case that appears in nearly every "List/Stack" interview round.

AI-Style Interview Questions

These are scenario-based / reasoning questions in the style modern AI-driven interview screens (HackerRank AI, Glider, etc.) tend to ask — they test judgment and trade-off thinking, not just recall.

You're designing a cache for a web app that needs frequent lookups by ID but rarely reorders elements. Would you choose ArrayList, LinkedList, or a Map — and why?

A Map (e.g. `HashMap<ID, Object>`) beats both Lists here since lookup-by-key is $O(1)$ average, versus $O(n)$ linear search in an ArrayList/LinkedList. Lists are the wrong data shape when the primary access pattern is "find by identifier" rather than "access by position."

Your production code throws `ConcurrentModificationException` intermittently under load. Walk through how you'd diagnose and fix it.

First confirm whether the collection is shared across threads or just mutated during its own iteration in single-threaded code (most common cause). Check for any remove/add calls inside a for-each or stream over the same list. Fix options: use `Iterator.remove()`, collect items to remove into a separate list and batch-remove after the loop, or move to a thread-safe/concurrent collection (`CopyOnWriteArrayList`, `ConcurrentHashMap`) if genuinely multi-threaded.

A junior developer used a Vector "to be safe" in a single-threaded batch job that's running slowly. What would you tell them?

Vector's synchronization is pure overhead in single-threaded code — every method call pays a lock cost for no benefit. Recommend switching to ArrayList; if thread-safety is genuinely needed later, wrap it explicitly rather than defaulting to a legacy synchronized class.

You need a data structure to implement "undo/redo" in a design tool. How would you model it using the classes discussed today?

Two stacks: an `undoStack` and a `redoStack` (`ArrayDeque` preferred over `java.util.Stack` for performance). Each action pushes onto `undoStack`; undo pops from `undoStack` and pushes onto `redoStack`; redo does the reverse. This mirrors the LIFO nature of both stacks.

Your ArrayList-based application suddenly slows down as data grows into the millions, especially on inserts near the front. Explain why, and propose a fix.

Inserting near index 0 forces an $O(n)$ shift of nearly the entire backing array every time, which becomes very expensive at scale. If front-insertions are common, a LinkedList (or `ArrayDeque` if only the very front/back matter) removes the shifting cost, trading it for pointer traversal cost only when random access is also needed.

How would you explain "amortized $O(1)$ " for `ArrayList.add()` to a non-technical stakeholder?

Most additions are instant, but every so often the list needs to "move to a bigger house" and copy everything over — that occasional big move is expensive, but averaged across all the fast additions in between, the typical cost per operation stays effectively constant.

Mock Interview Transcript

A realistic back-and-forth covering today's topics, as it might unfold in a real technical round.

INTERVIEWER

Let's start simple — what's the difference between Collection and Collections in Java?

CANDIDATE

Collection is the root interface for List, Set, and Queue — it defines the common contract like add, remove, and size. Collections, with an 's', is a utility class full of static helper methods, like sort and reverse, that operate on collection objects.

INTERVIEWER

Good. Now, if I asked you to store a list of 10,000 user IDs where I mostly append to the end and occasionally read by index, what would you pick and why?

CANDIDATE

ArrayList — appending at the end is $O(1)$ amortized, and index-based reads are $O(1)$ since it's backed by a plain array. LinkedList would only make sense if I were frequently inserting or deleting at the front.

INTERVIEWER

What happens under the hood when that ArrayList runs out of space?

CANDIDATE

It calls `grow()`, which allocates a new array roughly 1.5 times the size of the old one, then copies every existing element into it with `Arrays.copyOf`. That copy is $O(n)$, which is why `add()` is only "amortized" $O(1)$ rather than strictly $O(1)$.

INTERVIEWER

Interesting. Now suppose that same list is shared across two threads, one reading and one writing. Any concerns?

CANDIDATE

Yes — ArrayList isn't thread-safe. I could get a `ConcurrentModificationException`, or worse, corrupted internal state. I'd wrap it with `Collections.synchronizedList` and manually synchronize during iteration, or use `CopyOnWriteArrayList` if reads vastly outnumber writes.

INTERVIEWER

Why not just use Vector then, since it's already synchronized?

CANDIDATE

Vector synchronizes every individual method call, which adds locking overhead even when there's no actual contention, and it still doesn't make compound operations like check-then-add atomic. In practice, ArrayList plus an explicit concurrency strategy is more flexible and usually faster.

INTERVIEWER

Let's switch to LinkedList. When would you genuinely reach for it over ArrayList?

CANDIDATE

When I need frequent insertions or deletions at the head or tail — those are $O(1)$ on a LinkedList because it just relinks pointers, versus $O(n)$ shifting on an ArrayList. It's also a natural fit when I need Deque or Queue behavior, like a task queue or an undo history.

INTERVIEWER

One more — I have a Stack, and I call `remove(1)` where 1 is an Integer value I pushed. What actually gets removed?

CANDIDATE

That's a classic trap — since `Stack` extends `Vector` which extends `List`, `remove(int)` is overloaded. Passing the primitive `int 1` calls `remove(index)`, removing whatever's at position 1, not the value 1. To remove the value, I'd need to pass `Integer.valueOf(1)` so it resolves to `remove(Object)`.

INTERVIEWER

Nice catch. Last question — why does Oracle recommend `ArrayDeque` over `Stack` today?

CANDIDATE

`Stack` inherits from `Vector`, so it carries synchronization overhead it usually doesn't need, and it also exposes `List` methods like `insertElementAt` that let you violate LIFO ordering by accident. `ArrayDeque` is unsynchronized, purpose-built for stack and queue behavior, and generally faster.

INTERVIEWER

Great answers — that covers what I wanted for today's round.

CANDIDATE

Thank you — happy to go deeper into resizing internals or the fail-fast iterator mechanism if that's useful.

— End of Day 1 Notes · Java Collections Framework · Revise, Code, Repeat —